



Konzeption und Entwicklung eines KI-gestützten Agents zur automatisierten Generierung von CI/CD-Pipelines für CAP-Anwendungen auf Basis von Node.js

Praxisarbeit

aus dem Studiengang Wirtschaftsinformatik

an der Duale Hochschule Baden-Württemberg Mannheim

von

Tarnbir Singh

12.11.2025

Matrikelnummer, Kurs:	9997532, WWI23SEB
Unternehmen:	SAP SE, 69190, Walldorf
Betreuer im Unternehmen:	Niklas Miroll
Betreuer an der DHBW:	Prof. Dr. Sarah Detzler

Selbstständigkeitserklärung

Gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden- Württemberg vom 29.09.2017. Ich versichere hiermit, dass ich meine Arbeit mit dem Thema:

Konzeption und Entwicklung eines KI-gestützten Agents zur automatisierten Generierung von CI/CD-Pipelines für CAP-Anwendungen auf Basis von Node.js

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass alle eingereichten Fassungen übereinstimmen.

Walldorf 12.11.2025

Tarnbir Singh

Abstract

Die vorliegende Arbeit, „Konzeption und Entwicklung eines KI-gestützten Agents zur automatisierten Generierung von CI/CD-Pipelines für CAP-Anwendungen auf Basis von Node.js“, verfasst von Tarnbir Singh an der Dualen Hochschule Baden-Württemberg Mannheim in Kooperation mit der SAP SE, beschäftigt sich mit der Automatisierung von Konfigurationsprozessen in der Softwareentwicklung.

Ausgangspunkt ist die Beobachtung, dass die manuelle Konfiguration von CI/CD-Workflows ein zeitintensiver und fehleranfälliger Prozess ist, insbesondere bei domänenspezifischen Anwendungs-Frameworks wie dem SAP Cloud Application Programming Model (CAP). Diese erfordern oft Expertenwissen über komplexe, mehrstufige Build-Prozesse und spezifische Deployment-Ziele.

Methodisch folgt die Arbeit einem generativen Ansatz, der auf einem vortrainierten großen Sprachmodell (Large Language Model) basiert. Dieses wird durch einen detaillierten System-Prompt instruktiv gesteuert. Dafür werden die relevanten Metadaten der CAP-Anwendung zur Laufzeit aus dem Quell-Repository extrahiert und als dynamischer Kontext in den Prompt injiziert. Die Arbeit folgt dabei dem Phasenmodell CRISP-DM, von der Analyse der Datenbasis bis zur Implementierung des Agenten mittels SAP AI Core.

Die Evaluation des Prototyps erfolgt nicht quantitativ, sondern qualitativ durch ein Human-as-a-Judge-Verfahren. Die generierten Pipelines werden von Fachexpert:innen anhand der Kriterien funktionale Vollständigkeit (Lauffähigkeit), syntaktische Korrektheit und praktische Effizienz bewertet.

Die Ergebnisse zeigen, dass der Agent in der Lage ist, valide und praxistaugliche Pipelines zu generieren (Proof-of-Concept). Eine zentrale Erkenntnis ist, dass die Qualität des generierten Artefakts maßgeblich von der Vollständigkeit und Korrektheit der Eingangsdaten abhängt; Mängel in den analysierten Repositories mussten zunächst manuell behoben werden. Die Arbeit liefert einen fundierten Beleg für die Machbarkeit von generativen Agenten im DevOps-Kontext und bildet die Grundlage für weiterführende Automatisierungen.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation und Problemstellung	1
1.2	Zielsetzung und Forschungsfrage	2
1.3	Aufbau der Arbeit	2
2	Grundlagen	4
2.1	Continuous Integration und Continuous Delivery/Deployment	4
2.1.1	Continuous Integration (CI)	4
2.1.2	Continuous Delivery (CD)	5
2.1.3	Continuous Deployment (CD)	5
2.2	Cloud Application Programming Model	6
2.3	Cloud Foundry	7
2.4	KI-Agenten	9
2.5	Large Language Models	10
2.6	CRISP-DM Methodik	10
3	Business Understanding	14
4	Data Understanding	16
4.1	Explorative Datenanalyse	16
4.2	Datenqualität	17
5	Data Preparation	19
5.1	Datenkonsolidierung	19
5.2	Data Cleansing	19
5.3	Feature Engineering	20
6	Modeling	22
6.1	Modellverständnis und theoretische Einordnung	22
6.2	Systemarchitektur und Prozessmodell	23
6.2.1	User Interface Layer	23
6.2.2	Repository Analyzer	24
6.2.3	AI Core Integration Layer (LLM-Client)	24
6.2.4	Pipeline Validator	24
6.2.5	Deployment Manager	25
6.3	Modellierungsentscheidungen und Alternativen	25
6.3.1	Auswahl des Sprachmodells	25
6.3.2	Plattformwahl: SAP AI Core	26

6.3.3 CI/CD-Plattform: GitHub Actions	26
6.3.4 Programmiersprache und Technologie-Stack	27
6.3.5 Prompt Engineering und Kontext-Steuerung	27
6.4 Grenzen und Herausforderungen	28
6.5 Zusammenfassung der Modellierung und Übergang zur Evaluation	28
7 Evaluation	29
7.1 Evaluationsmethode: Human-as-a-Judge	29
7.2 Durchführung	31
7.3 Bewertung	31
7.4 Entscheidung über Deployability	32
8 Deployment	33
8.1 Zielplattform und Bereitstellung	33
8.2 Konfigurations- und Sicherheitsmanagement	34
8.3 Überwachung im Betrieb	34
9 Fazit und Ausblick	35
A System-Prompt	VI
Literaturverzeichnis	IX

Abbildungsverzeichnis

Abbildung 1	Entwicklung von traditionellen Infrastrukturen über IaaS und unstrukturierte Plattformen hin zu Cloud Foundry als strukturierte Cloud-Native Plattform [1].	8
Abbildung 2	Die sechs Phasen des CRISP-DM Prozessmodells und ihre iterativen Beziehungen [2].	11
Abbildung 3	Hierarchische Gliederung des CRISP-DM-Modells in vier Abstraktionsebenen [2].	12

1 Einleitung

1.1 Motivation und Problemstellung

Moderne Softwareentwicklung ist geprägt von kurzen Releasezyklen, steigenden Qualitätsanforderungen und einer zunehmenden Systemkomplexität. Um in diesem Umfeld Stabilität und Geschwindigkeit zu gewährleisten, haben sich Continuous-Integration- und Continuous-Delivery-Praktiken (CI/CD) als zentrale Bestandteile moderner Softwarebereitstellung etabliert. [3] Sie ermöglichen es, Änderungen automatisiert zu testen, zu integrieren und bereitzustellen - und damit Entwicklungsprozesse planbarer und reproduzierbarer zu gestalten.[4], [5]

Allerdings zeigen Studien und Praxiserfahrungen, dass die Einführung und Wartung solcher CI/CD-Prozesse mit erheblichem organisatorischem und technischem Aufwand verbunden ist. Der Aufbau von Pipelines erfordert ein tiefes Verständnis der jeweiligen Infrastruktur, der eingesetzten Frameworks und der Build-Tools. Zudem müssen diese Pipelines über ihren Lebenszyklus hinweg fortlaufend an neue Framework-Versionen, Sicherheitsrichtlinien oder geänderte Umgebungsparameter angepasst werden.[6], [7] Anders als klassische Softwaremodule sind CI/CD-Konfigurationen oft stark von betrieblichen Prozessen abhängig, was ihre Pflege besonders aufwendig und fehleranfällig macht.

Vor diesem Hintergrund wird Automatisierung zunehmend als Schlüssel gesehen, um Entwickler:innen von repetitiven Aufgaben zu entlasten und die Effizienz in der Softwarebereitstellung zu erhöhen. Ziel ist es dabei nicht, menschliche Expertise zu ersetzen, sondern Routinetätigkeiten zu standardisieren und dadurch Kapazitäten für konzeptionelle und kreative Tätigkeiten zu schaffen.[5] Aktuelle Fortschritte im Bereich generativer KI, insbesondere durch Large Language Models (LLMs), eröffnen neue Perspektiven für diese Form der Automatisierung. LLMs können auf Basis bestehender Projektstrukturen, Code-Patterns und Konfigurationsdateien eigenständig syntaktisch korrekte Artefakte erzeugen. Im Kontext von CI/CD bedeutet das die Möglichkeit, wiederkehrende Pipeline-Definitionen automatisch generieren zu lassen - mit dem Ziel, Konfigurationsaufwand zu reduzieren und gleichzeitig Konsistenz und Qualität der Deployments zu erhöhen.

Trotz zahlreicher Automatisierungswerkzeuge bleiben viele Aspekte der Pipeline-Erstellung weiterhin manuell und kontextabhängig. Framework-spezifische Strukturen, projektspezifische Abhängigkeiten und die hohe Varianz an Deployment-Szenarien verhindern oftmals eine vollständige Standardisierung. Klassische Automatisierungslösungen - etwa Skriptvorlagen oder Template-Engines - stoßen hier an ihre Grenzen, da sie vordefinierte Muster anwenden, aber keine inhaltliche Logik aus dem Projektkontext ableiten können.[7]

Genau an diesem Punkt setzt generative KI an: Durch Large Language Models (LLMs) wird es möglich, auf Basis von Repository-Inhalten wie Projektstruktur, Konfigurationsdateien und Abhängigkeitsdefinitionen eigenständig funktionsfähige Konfigurationsartefakte zu erzeugen. Der Einsatz eines solchen KI-gestützten Agenten eröffnet damit das Potenzial, die bisherige manuelle Arbeit bei der CI/CD-Konfiguration zu reduzieren und die Automatisierungstiefe deutlich zu erhöhen.

1.2 Zielsetzung und Forschungsfrage

Ziel dieser Arbeit ist es daher, zu untersuchen, ob sich ein KI-gestützter Agent realisieren lässt, der für Node.js-basierte CAP-Anwendungen auf der SAP BTP automatisch funktionsfähige CI/CD-Pipelines generiert. Im Mittelpunkt steht dabei nicht nur die technische Umsetzung, sondern auch die Bewertung, ob die erzeugten Artefakte praxistauglich sind.

Daraus leitet sich die zentrale Forschungsfrage dieser Arbeit ab:

Inwieweit lässt sich ein KI-Agent zur automatisierten Generierung von CI/CD-Pipelines für CAP-Anwendungen realisieren?

Zur Beantwortung dieser Frage wird ein Prototyp entwickelt, der Repository-Informationen analysiert, daraus eine Pipeline ableitet und diese als Pull Request bereitstellt. Die Praxistauglichkeit des Agenten wird anschließend in einem qualitativen Verfahren bewertet, um die Erreichung der Projektziele zu validieren.

1.3 Aufbau der Arbeit

Die Struktur dieser Arbeit folgt dem Vorgehensmodell des Cross-Industry Standard Process for Data Mining (CRISP-DM) [2]. Dieses Modell bildet den roten Faden, der den logischen Aufbau von der Problemdefinition bis zur fertigen Lösung vorgibt.

Nach der Fundierung der technischen Grundlagen (Abschnitt 2) wird in Abschnitt 3 (Business Understanding) das fachliche Problem spezifisch analysiert und die genauen Projektziele sowie die Erfolgskriterien definiert. Darauf aufbauend folgt der datengetriebene Prozess: Abschnitt 4 (Data Understanding) analysiert die Datenbasis während Abschnitt 5 (Data Preparation) die Bereinigung und Aufbereitung dieser Daten für die maschinelle Verarbeitung beschreibt. Den Kern der Arbeit bildet Abschnitt 6 (Modeling). Dieses Kapitel erläutert detailliert die Konzeption, die Systemarchitektur und die Implementierung des KI-Agenten, einschließlich des Prompt Engineerings und der Anbindung an den SAP AI Core. Abschließend wird der entwickelte Prototyp einer kritischen Prüfung unterzogen: Abschnitt 7 (Evaluation) bewertet die Ergebnisse anhand der definierten Erfolgskriterien. Abschnitt 8 (Deployment) beschreibt die Operationalisierung der Anwendung auf der Cloud Foundry Plattform. Die Arbeit schließt mit Kapitel 9 (Fazit und Ausblick), das die Ergebnisse im Kontext der Forschungsfrage zusammenfasst und zukünftige Entwicklungspotenziale aufzeigt.

2 Grundlagen

2.1 Continuous Integration und Continuous Delivery/Deployment

2.1.1 Continuous Integration (CI)

Continuous Integration (CI) ist eine zentrale Entwicklungspraxis in der Softwareentwicklung, bei der Entwickler ihre Code-Änderungen in kleinen, überschaubaren Inkrementen und in hoher Frequenz - oft mehrmals täglich - in ein zentrales, gemeinsames Repository überführen. Unmittelbar nach jeder dieser Integrationen wird ein vollautomatisierter Prozess, die sogenannte CI-Pipeline, angestoßen. Diese Pipeline kompiliert den gesamten Quellcode (erstellt einen „Build“) und führt eine umfassende Sequenz von automatisierten Tests aus.[8]

Das primäre Ziel dieses Vorgehens ist die frühzeitige Identifikation von Integrationsfehlern - also von Problemen, die entstehen, wenn die Code-Teile verschiedener Entwickler:innen zusammengeführt werden. Ohne kontinuierliche Integration würden solche Konflikte typischerweise erst in späten Entwicklungsphasen auffallen, was zu aufwendigen Debugging-Prozessen, längeren Release-Zyklen und einem erhöhten Risiko von Systeminstabilitäten führen kann. Durch das häufige Zusammenführen wird stattdessen eine schnelle Feedback-Schleife etabliert, die es Entwickler:innen ermöglicht, die Auswirkungen ihrer Änderungen auf das Gesamtsystem zeitnah zu verstehen und Anpassungen vorzunehmen, bevor größere Probleme entstehen. Dies trägt maßgeblich zu einer stabileren und wartungsfreundlicheren Codebasis bei.[9]

Die praktische Umsetzung erfolgt durch Werkzeuge wie Jenkins oder GitHub Actions, welche die Prozessautomatisierung steuern. Eine robuste CI-Pipeline stützt sich dabei auf verschiedene Testebenen, um eine durchgehende Qualitätssicherung zu gewährleisten. Diese umfassen Unit-Tests, welche die kleinsten, isolierten Komponenten der Software auf ihre korrekte Funktionsweise prüfen, Integrationstests, welche das reibungslose Zusammenspiel verschiedener Module verifizieren, und End-to-End-Tests, welche die gesamte Anwendung durch die Simulation vollständiger Benutzerszenarien aus der Perspektive des Endnutzers testen.[7]

2.1.2 Continuous Delivery (CD)

Continuous Delivery (CD) bezeichnet eine Entwicklungsdisziplin, bei der Software so konzipiert, gebaut und getestet wird, dass sie jederzeit produktiv auslieferbar ist. Nach Humbl liegt der Fokus darauf, die Software in einem kontinuierlich deployfähigen Zustand zu halten und sicherzustellen, dass jede Version der Anwendung auf Knopfdruck in eine beliebige Umgebung ausgerollt werden kann.[4]

Continuous Delivery baut auf Continuous Integration auf, erweitert diese jedoch um die finalen Stufen, die für eine produktionsreife Bereitstellung erforderlich sind. Dazu gehört, dass jedes Build-Artefakt automatisiert getestet und in produktionsähnliche Umgebungen übertragen wird, um die Funktionsfähigkeit sicherzustellen. Grundlage bildet dabei eine sogenannte Deployment Pipeline, welche die einzelnen Phasen von Build, Test und Release automatisiert abbildet.[4]

Das Ziel von Continuous Delivery ist es, das Risiko von Deployments zu reduzieren, die Nachvollziehbarkeit des Entwicklungsfortschritts zu erhöhen und schnelleres, verlässliches Feedback von Nutzern zu ermöglichen. Der wesentliche Unterschied zur Continuous Deployment besteht darin, dass Deployments bei Continuous Delivery zwar jederzeit möglich, aber nicht zwingend automatisiert erfolgen müssen. Die Entscheidung über den produktiven Rollout liegt letztlich beim Team oder dem jeweiligen Fachbereich.[9]

2.1.3 Continuous Deployment (CD)

Continuous Deployment beschreibt den Prozess, bei dem Code-Änderungen nach erfolgreicher Validierung in der Continuous Integration-Pipeline automatisch bis in die Produktionsumgebung ausgerollt werden. Damit schließt Continuous Deployment unmittelbar an Continuous Delivery an und führt dessen Prinzipien zu Ende.[10]

Der gesamte Übergang vom Code-Commit bis zum produktiven Release erfolgt hierbei ohne manuelle Eingriffe, wodurch menschliche Fehler reduziert und der Bereitstellungsprozess beschleunigt werden. Ziel ist es, Änderungen schnell, sicher und mit minimalem Overhead in den produktiven Betrieb zu überführen.[10]

Automatisierte Deployment-Pipelines bilden den Kern dieses Prozesses. Sie verifizieren, dass jede Version der Anwendung jederzeit in einem auslieferbaren Zustand ist. Dadurch wird eine kurze Time-to-Market ermöglicht, da neue Funktionen unmittelbar nach der Integration

bereitgestellt werden können. Continuous Deployment ist somit die konsequenteste Form der Automatisierung im CI/CD-Kontext: Jede Änderung, die die Qualitätskriterien erfüllt, wird produktiv gestellt.[9]

Im Gegensatz dazu belässt Continuous Delivery die Entscheidung über den Rollout beim Menschen. Continuous Deployment überträgt diese Entscheidung vollständig an die Pipeline selbst und realisiert damit einen durchgängigen, automatisierten Auslieferungsprozess.[9]

2.2 Cloud Application Programming Model

Das SAP Cloud Application Programming Model (CAP) ist ein Framework aus Sprachen, Bibliotheken und Werkzeugen, das zur Entwicklung von unternehmenstauglichen Cloud-Diensten und -Anwendungen dient. Es leitet Entwickler:innen entlang eines sogenannten „goldenen Pfads“ von bewährten Best Practices. Ziel ist es, Entwickler:innen von repetitiven, technischen Aufgaben zu entlasten, sodass sie sich auf die eigentliche Fachdomäne konzentrieren können.[11]

Der technische Eckpfeiler von CAP ist Core Data Services (CDS). CDS ist eine deklarative Sprache, die es ermöglicht, Datenmodelle und Service-Definitionen auf einer konzeptionellen Ebene zu erfassen [11]. Diese CDS-Modelle dienen als zentrale Quelle und werden von dort aus in native Artefakte für verschiedene Schichten übersetzt - beispielsweise in SQL-Datenbankschemata für die Persistenzschicht oder in OData-Services für die Applikationsschicht.[12]

CAP folgt dem Prinzip „Convention over Configuration“, was bedeutet, dass das Framework sinnvolle Standardeinstellungen (Konventionen) vorgibt. Entwickler:innen müssen dadurch nicht jede Kleinigkeit selbst konfigurieren, was den initialen Einrichtungsaufwand reduziert. Zudem ist es „agnostisch“ konzipiert; es ist also nicht fest an eine bestimmte Technologie gebunden. Dies erlaubt es beispielsweise, in der lokalen Entwicklung eine einfache SQLite-Datenbank zu verwenden, während in der Produktion SAP HANA zum Einsatz kommt, ohne dass der Anwendungscode geändert werden muss. [13] Das Framework unterstützt primär die Runtimes Node.js und Java und bietet für das Deployment eine nahtlose Integration in die Cloud Foundry Runtime.[12]

2.3 Cloud Foundry

Cloud Foundry ist eine Cloud-Native Plattform, die Anwendungen abstrahiert von der zugrundeliegenden Infrastruktur betreibt und dabei viele betriebliche Aufgaben automatisiert übernimmt. Ziel ist es, Entwickler:innen zu entlasten und die Bereitstellung von Software zu beschleunigen.[14]

Eine Cloud-Native Plattform integriert Funktionen wie Resilienz, Benutzerverwaltung und Logging und reduziert damit den Konfigurationsaufwand. Cloud Foundry wird dabei als structured, opinionated und open beschrieben: Sie bietet eingebaute, konsistente Funktionen (structured), folgt bewährten Prinzipien und reduziert Komplexität (opinionated), und ist offen für verschiedene Infrastrukturen, Programmiersprachen sowie Frameworks (open).[1]

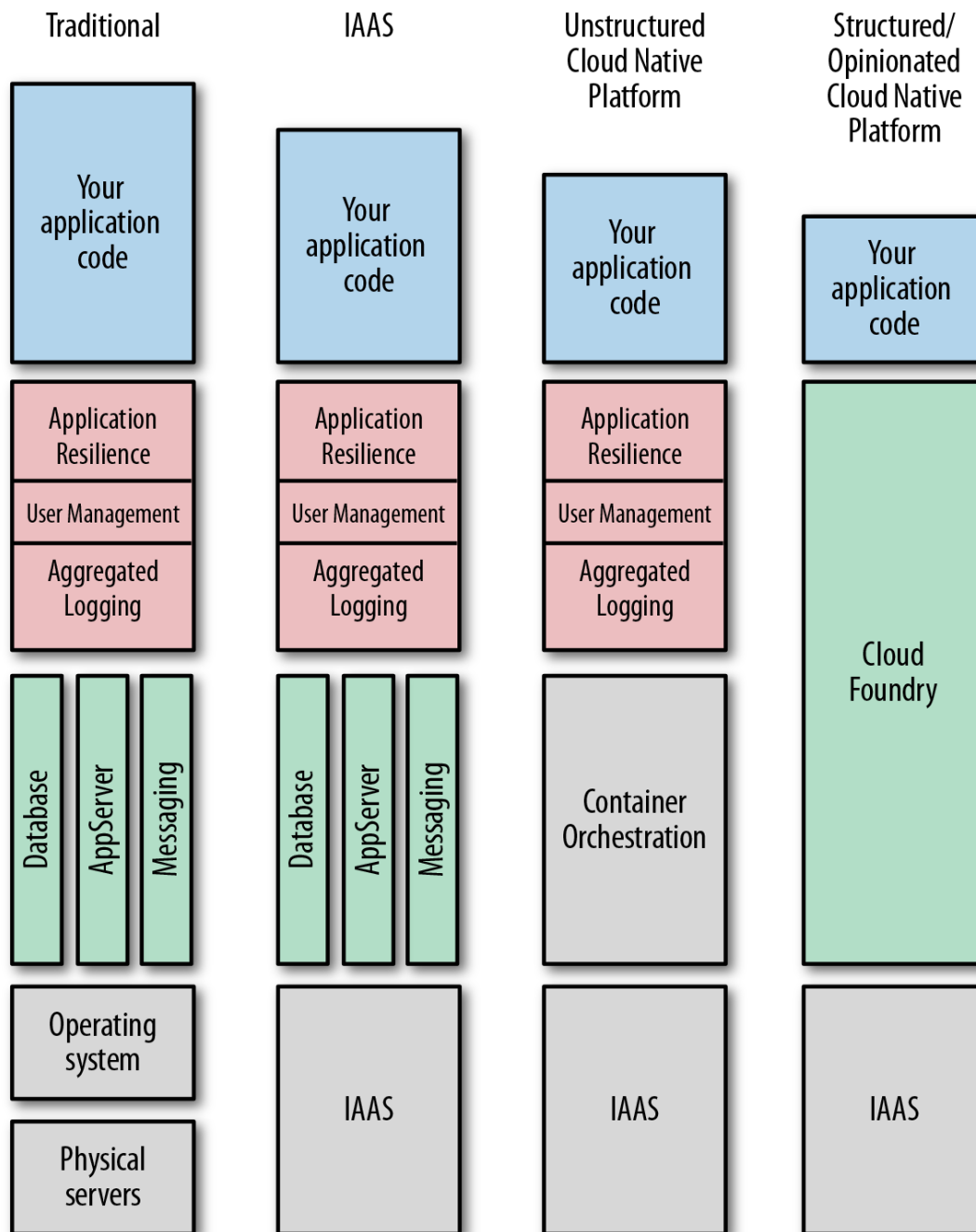


Abbildung 1 — Entwicklung von traditionellen Infrastrukturen über IaaS und unstrukturierte Plattformen hin zu Cloud Foundry als strukturierte Cloud-Native Plattform [1].

Die Abbildung 1 verdeutlicht die Unterschiede zwischen traditionellen Ansätzen, IaaS, unstrukturierten Plattformen und Cloud Foundry. Während in klassischen Modellen zahlreiche Schichten manuell verwaltet werden müssen, integriert Cloud Foundry zentrale Funktionen und hebt so die Abstraktionsebene deutlich an.

Ein zentrales Konzept ist die Entlastung von undifferentiated heavy lifting tasks, also grundlegenden, aber nicht differenzierenden Infrastrukturarbeiten wie Skalierung oder Logging. Cloud Foundry übernimmt diese Aufgaben automatisch und ermöglicht es, dass sich Entwickler:innen auf die eigentliche Geschäftslogik konzentrieren können. Die Plattform agiert damit wie ein Cloud-Betriebssystem, das Anwendungen konsistent und resilient betreibt.[1]

2.4 KI-Agenten

Die Entwicklung von KI-gestützten Systemen hat sich über das reine Prompt Engineering hinaus entwickelt, da komplexe Aufgaben Planung und Validierung erfordern [15]. In diesem Kontext hat sich das Konzept des KI-Agenten als zentraler Baustein etabliert.

Im Kontext moderner Large Language Models (LLMs) wird der Begriff „Agent“ breiter gefasst als in der klassischen KI (z.B. Reinforcement Learning). Ein Agent ist hierbei ein System, das seine Umgebung wahrnimmt, Entscheidungen trifft und über Aktoren auf diese Umgebung einwirkt, wobei das LLM als „leitende Intelligenz“ zur Erreichung eines Ziels dient.[15]

Nach (Lanham) werden Agenten primär durch ihren Autonomiegrad klassifiziert. Die Skala reicht von Agenten-Proxys, die lediglich Nutzereingaben optimieren, über Assistenten, die für Aktionen eine explizite Nutzerfreigabe benötigen, bis hin zu autonomen Agenten, welche eine Anfrage interpretieren, daraufhin selbstständig einen Plan und die notwendigen Schritte zur Zielerreichung ohne weitere Freigaben ausführen.[15]

Die Systemarchitektur solcher Agenten lässt sich nach (Lanham) in fünf Kernkomponenten gliedern, die auch die theoretische Basis für das in dieser Arbeit entwickelte Modell bilden: Das Fünf-Komponenten-Modell nach Lanham umfasst die folgenden zentralen Elemente:

- **Profil (Persona):** Definiert als System Prompt die Rolle und das Ziel des Agenten.
- **Aktionen und Werkzeuge (Actions/Tool Use):** Umfassen die Fähigkeiten zur Interaktion mit der Umgebung, etwa API-Aufrufe oder den Zugriff auf externe Systeme.
- **Wissen und Gedächtnis (Knowledge/Memory):** Stellt den für die Aufgabenerfüllung erforderlichen Kontext bereit, beispielsweise in Form von Umgebungs- oder Projektdaten.
- **Schlussfolgern und Bewerten (Reasoning/Evaluation):** Beschreibt die interne Logik zur Problemlösung und Qualitätsprüfung der generierten Ergebnisse.
- **Planung (Planning):** Definiert den übergeordneten Workflow und steuert die Ausführung zur Erreichung des Ziels. [15]

2.5 Large Language Models

Ein **Large Language Model (LLM)** ist ein auf künstlicher Intelligenz basierendes Modell, das in der Lage ist, große Mengen an Textdaten zu analysieren und darauf aufbauend Texte in natürlicher Sprache zu generieren [16]. Diese Modelle nutzen Deep-Learning-Methoden, insbesondere den sogenannten Transformer-Ansatz, um Sprache zu verstehen, zu verarbeiten und neue Inhalte zu erzeugen.[17]

Der Transformer-Ansatz stellt eine spezialisierte Architektur des maschinellen Lernens dar, die auf dem Prinzip der Selbstaufmerksamkeit (Self-Attention) basiert. Diese ermöglicht es dem Modell, Beziehungen zwischen Wörtern, Sätzen und Kontexten zu erkennen und semantische Abhängigkeiten über lange Textabschnitte hinweg abzubilden.[18]

Dadurch kann ein LLM sprachliche Muster effizient erfassen und kontextsensitiv verarbeiten.[19]

Technisch basieren LLMs auf tiefen neuronalen Netzen, die - inspiriert vom Aufbau und der Funktionsweise des menschlichen Gehirns - Informationen in mehreren Schichten von künstlichen Neuronen verarbeiten. Diese hierarchische Struktur erlaubt es, Bedeutungen und Zusammenhänge in Sprache abzuleiten und zu generalisieren.[19], [20]

Insgesamt ermöglichen LLMs das Verständnis, die Interpretation und die Generierung von Texten, die in Stil, Struktur und Inhalt stark an menschliche Sprache angelehnt sind. Damit stellen sie die Grundlage vieler moderner generativer KI-Systeme dar, wie Chatbots, Textgeneratoren oder KI-gestützte Assistenzsysteme.[16]

2.6 CRISP-DM Methodik

Die in dieser Arbeit angewandte Methodik zur Entwicklung des KI-gestützten Agents basiert auf dem CRISP-DM (**C**Ross-**I**ndustry **S**tandard **P**rocess for **D**ata **M**ining) Prozessmodell. CRISP-DM wurde 1999 veröffentlicht, um einen branchenübergreifenden Standard für Data-Mining-Projekte zu etablieren, und gilt heute als eine der am weitesten verbreiteten Vorgehensweisen in diesem Bereich. Das Modell strukturiert den Prozess in sechs Phasen, die zwar sequenziell dargestellt werden, in der Praxis jedoch häufig iterativ durchlaufen werden, was Rücksprünge zu früheren Phasen einschließt.[2]



Abbildung 2 – Die sechs Phasen des CRISP-DM Prozessmodells und ihre iterativen Beziehungen [2].

Den Ausgangspunkt des Prozesses bildet das **Geschäftsverständnis (Business Understanding)**, in dem die fachlichen Projektziele und Anforderungen definiert und in eine konkrete Problemstellung sowie einen vorläufigen Plan überführt werden. Auf dieser Grundlage folgt das **Datenverständnis (Data Understanding)**, welches die initiale Erfassung und Analyse der Daten zum Ziel hat; hierbei werden deren Eigenschaften, Qualität und Relevanz für das Projektziel bewertet. In der anschließenden **Datenvorbereitung (Data Preparation)** werden die Rohdaten durch Verfahren wie die Datenkonsolidierung, Bereinigung (Data Cleansing) und Formatierung (Feature Engineering) in eine für die Modellierung geeignete

Form gebracht. Die Phase der **Modellierung (Modeling)** umfasst daraufhin die Auswahl und Anwendung verschiedener Techniken zur Erstellung der Modelle, wobei deren Parameter iterativ zur Leistungssteigerung optimiert werden. Im Rahmen der **Evaluation** werden diese Modelle einer kritischen Prüfung unterzogen, um ihre Qualität und ihren Beitrag zur Erreichung der Geschäftsziele zu validieren. Den Abschluss des Zyklus bildet das **Deployment**, bei dem das erfolgreich validierte Modell in die operative Umgebung integriert wird, um die daraus resultierenden Erkenntnisse den Endanwendern bereitzustellen.[2]

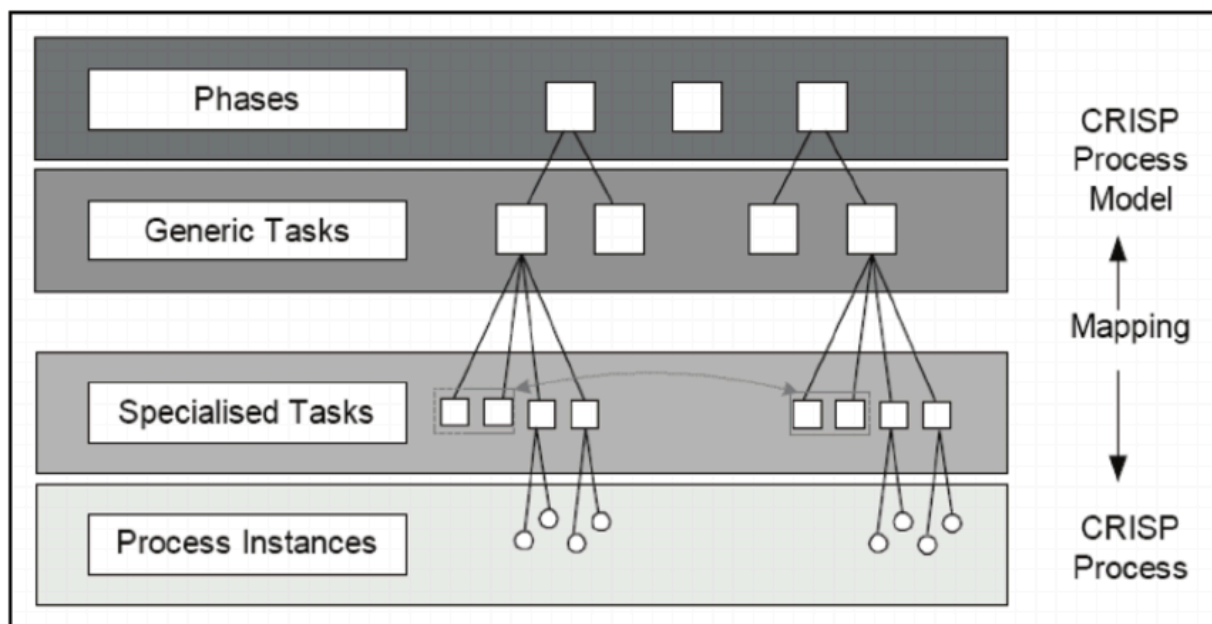


Abbildung 3 – Hierarchische Gliederung des CRISP-DM-Modells in vier Abstraktionsebenen [2].

Der hierarchische Aufbau des CRISP-DM-Prozesses ist der Schlüssel zu seiner Flexibilität in der Praxis. Das Modell gliedert ein Projekt in mehrere Ebenen - von der übergeordneten Strategie bis hin zu konkreten, operativen Aufgaben (siehe Abbildung 3). Diese Struktur erlaubt es, das Vorgehen passgenau an die spezifischen Anforderungen und die Komplexität jedes einzelnen Vorhabens anzupassen: [2]

- **Allgemeine Aufgaben (Generic Tasks):** Diese beschreiben die grundlegenden Aktionen innerhalb jeder Phase. In dieser Arbeit wären das beispielsweise die „Analyse der Projektstruktur“ oder die „Generierung von Pipeline-Konfigurationen“.

- **Spezialisierte Aufgaben (Specialized Tasks):** Hier werden die allgemeinen Aufgaben an die konkreten Gegebenheiten des Projekts angepasst. Aus der Analyse der Projektstruktur wird beispielsweise die spezifische Aufgabe „Parsen der package.json-Datei zur Identifikation von Node.js-Abhängigkeiten“. Aus der Generierung von Konfigurationen wird die Aufgabe „Erstellung einer mta.yml-Datei für das Deployment auf Cloud Foundry“.
- **Prozessinstanzen (Process Instances):** Diese Ebene dokumentiert die tatsächliche Durchführung des Projekts. Sie erfasst spezifische Entscheidungen, gewählte Parameter und erzielte Ergebnisse und bildet somit ein detailliertes Protokoll des Projektverlaufs.[2]

3 Business Understanding

Der Anlass für dieses Projekt und somit auch für die vorliegende Arbeit entstand aus dem Bestreben der Abteilung für Kundenberatung rund um die SAP Business Technology Platform (BTP), den internen Aufwand bei der Erstellung von CI/CD-Pipelines für CAP-Applikationen zu reduzieren. Besonders im Kontext von Kunden-Demonstrationen, bei denen regelmäßig neue CAP-Projekte erstellt werden, zeigte sich, dass die manuelle Konfiguration der notwendigen Pipeline-Dateien zeitintensiv, fehleranfällig und wenig skalierbar ist. Unsere internen Beobachtungen decken sich mit Ergebnissen aus der Literatur, wonach manuell konfigurierte CI/CD-Prozesse fehleranfällig und ineffizient sein können [21].

Interne Erfahrungen bestätigten, dass ein Großteil der Arbeitszeit für wiederkehrende, manuelle Konfigurationsaufgaben aufgewendet wird, obwohl diese Aufgaben ein hohes Automatisierungspotenzial aufweisen. Studien zeigen, dass etwa 60% der IT-Tätigkeiten aus solchen Routineaufgaben bestehen, die mit modernen Automatisierungstechnologien effizienter und qualitativ konsistenter ausgeführt werden könnten. Die Automatisierung von Prozessen führt dabei nicht nur zu einer signifikanten Reduktion menschlicher Fehler, sondern schafft auch Freiräume für kreative und strategische Tätigkeiten. [22]

Vor diesem Hintergrund entstand die Idee und das Ziel, einen KI-gestützten Agenten zu entwickeln, der die Erstellung von CI/CD-Pipelines automatisiert. Ein solcher Ansatz ist sowohl für die SAP SE als auch für die beteiligten Abteilungen von strategischem Interesse, da er Effizienzgewinne, Qualitätssteigerungen und Ressourceneinsparungen verspricht. Durch die Automatisierung der Pipeline-Generierung lassen sich Standardprozesse beschleunigen und konsistente Deployment-Konfigurationen gewährleisten, was letztlich die Time-to-Cloud für CAP-Projekte reduziert.

Die angestrebte Lösung mittels eines KI-Agenten soll Entwickler:innen gezielt bei der Arbeit unterstützen, indem sie den manuellen Aufwand minimiert und technische Details der Pipeline-Konfiguration übernimmt. Ziel ist es, dass der Agent lediglich die GitHub-Repository-URL eines Projekts entgegennimmt, relevante Parameter und Dateien automatisch analysiert und daraus eine funktionsfähige CI/CD-Pipeline generiert.

Diese soll als Pull Request bereitgestellt werden, sodass Entwickler:innen sie vor der Integration prüfen und anpassen können. Der Agent agiert damit als intelligentes Assistenzsystem, das die Produktivität erhöht, ohne den Menschen aus dem Prozess auszuschließen. In dieser Pull Request können dann Entwickler:innen durch GitHub Actions die generierte Pipeline validieren und bei Bedarf modifizieren.

Zur Beurteilung des Agenten wird ein qualitatives, expertenbasiertes Verfahren (Human-as-a-Judge) eingesetzt, bei dem die generierten Pipelines durch mich sowie erfahrene Entwickler:innen der Abteilung bewertet werden. Der Erfolg bemisst sich dabei an drei zentralen, praxisnahen Kriterien: funktionale Vollständigkeit (die Pipeline lässt sich erfolgreich ausführen), syntaktische Korrektheit (valide YAML-Struktur) und praktische Effizienz (keine unnötigen Build- oder Deployment-Schritte). Eine detaillierte Ausgestaltung und methodische Begründung dieses Bewertungsverfahrens erfolgt im Evaluationskapitel.

4 Data Understanding

Im Rahmen des CRISP-DM-Modells dient die Phase des Data Understanding der initialen Analyse und Bewertung der zugrundeliegenden Datenbasis, um deren Struktur, Qualität und Eignung für den definierten Anwendungsfall zu verstehen. Im Kontext dieser Arbeit handelt es sich bei den „Daten“ nicht um tabellarische Datensätze, sondern um komplexe Software-Artefakte in Form von fünf GitHub-Repositories, die jeweils eine CAP-Anwendung repräsentieren. Die relevanten Merkmale für die Analyse durch den KI-Agenten sind dabei die Verzeichnisstruktur, Inhalte von Konfigurationsdateien wie *package.json* und der *mta.yaml*, Build-Skripte sowie versionsspezifische Unterschiede der Projekte.

Als Datenbasis wurde eine Auswahl von fünf exemplarischen CAP-Anwendungen herangezogen. Diese Anzahl ergibt sich aus den im Rahmen dieser Ausarbeitung zur Verfügung stehenden, relevanten Projekten. Vier davon - **CAP-SFlight**, **CAP-Orders**, **CAP-Reviews** sowie **CAP-Bookshop** - sind öffentlich verfügbare Schulungsanwendungen von SAP. Diese wurden aufgrund ihrer einfachen und übersichtlichen Struktur, des erleichterten Zugriffs sowie der öffentlichen Validierung ihrer Funktionsfähigkeit als primäre Testfälle für den KI-Agenten gewählt. Die Auswahl dieser grundlegenden Anwendungsfälle soll zunächst sicherstellen, dass der Agent die Standardkonfigurationen von CAP-Projekten korrekt verarbeiten kann, bevor er mit komplexeren, nicht-standardisierten Repositories konfrontiert wird.

Die fünfte Anwendung ist eine interne **Retrieval Augmented Generation (RAG)**-Anwendung, die eine vergleichbar aufgebaute, jedoch deutlich komplexere und umfangreichere Struktur aufweist. Sie dient als finaler Testfall, um die Leistungsfähigkeit des KI-Agenten auch in komplexeren CAP-Szenarien zu evaluieren. Alle für diese Ausarbeitung genutzten CAP-Anwendungen sind prinzipiell voll funktionsfähig und für ein Deployment auf der Cloud Foundry Runtime der SAP BTP ausgelegt.

4.1 Explorative Datenanalyse

Im Rahmen der explorativen Analyse wurden die fünf CAP-Anwendungen systematisch untersucht, um Gemeinsamkeiten, Muster und relevante Unterschiede für die automatisierte CI/CD-Pipeline-Generierung zu identifizieren.

Die Analyse bestätigte, dass alle Anwendungen auf dem Cloud Application Programming Model basieren und Node.js als Laufzeitumgebung nutzen. Es konnte eine durchgängige Verwendung von SAP HANA Cloud als Datenbank sowie XSUAA für die Authentifizierung festgestellt werden. Die Projekte folgen der typischen CAP-Struktur mit den Kernverzeichnissen *db/*, *srv/* und *app/* und verwenden standardisierte Build- und Deploy-Befehle. Diese technologische Homogenität schafft eine valide Grundlage für die automatisierte Analyse durch den Agenten.

Gleichzeitig traten signifikante Unterschiede zutage, die für die Konzeption des Agenten von entscheidender Bedeutung sind. Wie in der nachfolgenden Tabelle ersichtlich, variieren die eingesetzten CAP- und Node.js-Versionen, was zu Unterschieden in der Command Line Interface (CLI) und den Build-Prozessen führen kann. Zudem weisen die Projekte eine unterschiedliche Komplexität auf, die sich in der Anzahl der Abhängigkeiten, dem Vorhandensein von UI-Komponenten oder der Integration zusätzlicher Services (z. B. AI SDKs) manifestiert.

Merkmal	CAP-Bookshop	CAP-SFlight	CAP-Orders	CAP-Reviews	CAP-RAG
CAP Version	v6.3.0	v5.9.8	v7.1.0	v7.1.0	v8.0.1
Node.js Version	16.x	14.x	18.x	18.x	18.x
DB-Technologie	HANA Cloud	SQLite	HANA Cloud	HANA Cloud	HANA Cloud
<i>mta.yaml</i> vorhanden?	Nein	Nein	Nein	Ja	Ja
UI-Komponente	Fiori UI	Keine	Keine	Fiori UI	UI5 Freestyle
Zusätzl. Services	Keine	Keine	Keine	Keine	AI SDK, XSUAA
Anzahl Dependencies	25	18	22	23	42

Tabelle 1 – Überblick über die CAP-Anwendungen

4.2 Datenqualität

Die Untersuchung der Datenqualität zeigte mehrere Mängel in den betrachteten Repositories, die ein unmittelbares Deployment auf die Cloud Foundry Runtime ohne Anpassungen

verhinderten. Aufgrund der begrenzten Stichprobe kann jedoch nicht von einem generellen Qualitätsproblem bei CAP-Anwendungen ausgegangen werden.

Vollständigkeit: Bei drei der fünf Repositories fehlte die *mta.yaml*-Datei. Diese ist jedoch essenziell für das Deployment auf die Cloud Foundry Runtime, da sie die gesamte Anwendungsarchitektur als Multi-Target Application (MTA) definiert.

Korrektheit: In einigen Projekten waren Build-Skripte in der *package.json*-Datei veraltet oder fehlerhaft, was zu Fehlern im Build-Prozess geführt hätte.

Konsistenz: Es wurden unterschiedliche Node.js-Versionen und CAP-Versionen identifiziert, was eine flexible Pipeline-Logik erfordert, die diese Unterschiede berücksichtigen kann.

Die Behebung und Aufarbeitung der dieser Mängel wird im nächsten Kapitel, der Data Preparation (Abschnitt 5), detailliert behandelt.

Zusammenfassend lässt sich festhalten, dass die analysierten Projekte zwar eine einheitliche technologische Grundlage bieten, sich jedoch in Komplexität, Vollständigkeit und Konfigurationsstand erheblich unterscheiden. Das daraus gewonnene Verständnis bildet die entscheidende Basis für das nächste Kapitel, die Datenvorbereitung, sowie für die Entwicklung des KI-Agenten. Insbesondere muss der Agent in der Lage sein, projektspezifische Konfigurationen zu interpretieren und seine Prozesse entsprechend anzupassen.

5 Data Preparation

Aufbauend auf der im vorherigen Kapitel durchgeführten explorativen Analyse und der Aufarbeitung von Qualitätsmängeln werden in diesem Kapitel die Schritte zur systematischen Aufbereitung der Datenbasis beschrieben. Ziel dieser Phase ist es, eine fehlerfreie, konsistente und für den KI-Agenten optimal nutzbare Grundlage zu schaffen. Der Prozess folgt den etablierten Phasen des CRISP-DM-Modells: Datenkonsolidierung, Data Cleansing und Feature Engineering (siehe Grundlagenkapitel CRISP-DM).

5.1 Datenkonsolidierung

Zunächst habe ich die fünf GitHub-Repositories, die die Datenquellen dieser Arbeit darstellen, in eine kontrollierte und einheitliche Umgebung überführt. Dies geschah, indem für jede Anwendung ein privater Fork auf einem dedizierten GitHub-Account angelegt wurde. Dieser Schritt war strategisch erforderlich, um zum einen die **Kontrolle** über die für die Bereinigung notwendigen Änderungen zu erhalten, zum anderen die **Sicherheit** sensibler Anmeldeinformationen durch GitHub Actions Secrets zu gewährleisten und schließlich den programmatischen **Zugriff** für den Agenten mittels eines dedizierten Tokens zu ermöglichen.

5.2 Data Cleansing

Anschließend wurden die im Data Understanding identifizierten Mängel in der Datenqualität behoben, um eine durchgängige Funktions- und Deploy-Fähigkeit der Anwendungs-Repositories sicherzustellen. Dies umfasste die manuelle Erstellung der fehlenden *mta.yaml*-Dateien, die für ein Deployment auf die Cloud Foundry Runtime essenziell sind, sowie die Korrektur veralteter oder fehlerhafter Build-Skripte in den *package.json*-Dateien.

Eine automatisierte Generierung der *mta.yaml* durch den Agenten wurde bewusst ausgeschlossen. In dieser Konfigurationsdatei werden anwendungsspezifische Architekturentscheidungen, wie die Ressourcenallokation, getroffen, die maßgeblichen Einfluss auf Performance und Kosten haben. Solche kritischen Design-Entscheidungen sollen von einem Entwickler verifiziert werden, weshalb der Agent den Prozess lediglich unterstützen, nicht aber vollständig autonom durchführen soll.

5.3 Feature Engineering

Der letzte und für das Modeling zentrale Schritt war das Feature Engineering. Dabei geht es darum, aus den umfangreichen Rohdaten der Repositories jene Merkmale (Features) auszuwählen, die für die nachgelagerte Modellerzeugung relevant sind. Im Gegensatz zu klassischem Feature Engineering, das häufig Transformationen numerischer oder kategorischer Werte umfasst, besteht der Fokus hier in der gezielten Reduktion der Repository-Daten auf inhaltlich relevante Artefakte, die das LLM benötigt, um eine valide CI/CD-Pipeline generieren zu können.

Basierend auf den Erkenntnissen aus dem Data Understanding wurde definiert, dass zur Generierung einer CI/CD-Pipeline gezielt drei zentrale Merkmale (Features) aus jedem Repository extrahiert werden müssen:

1. Die Projektstruktur in Form der Ordner- und Dateihierarchie.
2. Der Inhalt der Konfigurationsdatei *package.json*.
3. Der Inhalt des Deployment-Deskriptors *mta.yaml*.

```
RELEVANT_FILES = ["mta.yaml", "package.json"]

def get_relevant_files(self):
    """Lädt relevante Dateien aus dem Repository."""
    collected = {}
    url = f"{self.base_url}/repos/{self.owner}/{self.repo}/contents"
    items = self._request("GET", url)

    for item in items:
        if item["name"] in RELEVANT_FILES and item.get("download_url"):
            file_resp = requests.get(item["download_url"])
            if file_resp.status_code == 200:
                collected[item["name"]] = file_resp.text
    return collected
```

Quelltext 1 – Python-Funktion zur Extraktion relevanter Dateien via GitHub API.

Die Extraktion erfolgt durch gezielte Leseoperationen (siehe Quelltext 1) über die GitHub API, autorisiert durch den zuvor konfigurierten Token. Das Ergebnis dieses Prozesses ist ein Objekt, welches alle relevanten Dateinamen und deren Inhalte bereithält.

Nach Abschluss der Datenvorbereitung liegt somit eine bereinigte und konsistente Datenbasis vor. Die fünf CAP-Anwendungen sind nun funktionsfähig und ein definierter Satz an Merkmalen wurde als Input für den Agenten extrahiert. Diese aufbereitete Grundlage ist die unmittelbare Voraussetzung für die im folgenden Kapitel beschriebene Phase des **Modelings**, in der der KI-Agent entwickelt wird, um diesen Input zu verarbeiten und die CI/CD-Pipeline zu generieren.

6 Modeling

Im Rahmen des CRISP-DM-Modells beschreibt die Phase des Modeling die Konzeption, Komponentenauswahl und Implementierung des KI-gestützten Agenten. Ziel dieses Kapitels ist es, die Architektur und Funktionsweise des entwickelten Systems zu erläutern sowie die zugrunde liegenden Entscheidungen und betrachteten Alternativen nachvollziehbar zu begründen. Damit leistet das Kapitel einen zentralen Beitrag zur Beantwortung der Forschungsfrage, inwieweit sich ein KI-Agent zur automatisierten Generierung von CI/CD-Pipelines für CAP-Anwendungen realisieren lässt.

6.1 Modellverständnis und theoretische Einordnung

Um das Modell methodisch korrekt im Rahmen von CRISP-DM zu verorten, wird das in dieser Arbeit behandelte Modell gegenüber klassischen Machine-Learning- und Data-Mining-Ansätzen abgegrenzt. Dafür benötigt es zunächst eine klare Definition und Einordnung dieser Begriffe.

Klassisches Machine Learning (ML) zielt nach Zeigermann und Nguyen primär auf analytische oder prädiktive Aufgaben ab, etwa durch Supervised, Unsupervised oder Reinforcement Learning [23]. Data Mining bezeichnet nach Shetty et al. die Anwendung von KI-Strategien auf große Datenmengen zur Mustererkennung [24].

Das in dieser Arbeit modellierte System unterscheidet sich grundlegend von klassischen Machine-Learning-Ansätzen. Sein Ziel ist generativ statt prädiktiv: Es trifft keine Vorhersagen über bestehende Daten, sondern erzeugt ein gänzlich neues Code-Artefakt - konkret eine *workflow.yml*-Datei. Dieser generative Ansatz wurde gewählt, da die Alternative - ein klassisches, datengetriebenes Modell - eine umfangreiche Menge an gelabelten Daten (d.h. Repository-Zustände gepaart mit zugehörigen, korrekten Pipeline-Dateien) für ein Training erfordert hätte. Eine solche Datenbasis stand für dieses Projekt nicht zur Verfügung.

Der Prozess basiert daher nicht auf einem Training. Zwar werden strukturierte Daten aus dem Repository (z.B. *package.json*, *mta.yaml*) verarbeitet, diese dienen jedoch nicht als Trainingsdaten. Sie werden stattdessen zur Laufzeit als dynamischer Kontext in den System-Prompt injiziert, um ein vortrainiertes LLM instruktiv auf den spezifischen CAP-Kontext zu steuern.

Der Anwendungsbereich ist damit spezifisch - er konzentriert sich auf die Analyse und Verarbeitung einzelner CAP-Repositories - und nicht explorativ. Ein klassisches Machine-Learning-Modell wäre für diesen Anwendungsfall ungeeignet, da weder ausreichend Trainingsdaten vorliegen noch ein prädiktives Ziel existiert: Es soll kein numerischer Wert geschätzt oder ein Label vorhergesagt werden, sondern ein konkretes Artefakt generiert werden. Zudem ist der Zieloutput deterministisch - die generierte Pipeline folgt klaren technischen Regeln, die aus Projektstruktur und Metadaten abgeleitet werden können.

Ebenso sind Data-Mining-Ansätze nicht anwendbar, da diese auf die Identifikation statistischer Muster in sehr großen Datenmengen ausgerichtet sind. Im vorliegenden Kontext existieren weder große Datenmengen noch ein explorativer Erkenntnisgewinn über latente Strukturen. Stattdessen steht die regelbasierte Ableitung einer Pipeline aus wenigen, klar strukturierten Artefakten im Vordergrund.

Das entwickelte System lässt sich daher als **regelbasierter, kontextadaptiver, jedoch nicht lernfähiger KI-Agent** beschreiben. Es interpretiert die bereitgestellten Repository-Daten, leitet daraus deterministische Aktionen ab und überprüft die Ergebnisse durch eine integrierte Validierung. Damit handelt es sich nicht um ein lernendes Modell im klassischen Sinne, sondern um einen intelligenten, generativen Agenten, der auf einem LLM basiert und Aufgaben kontextbezogen ausführt.

6.2 Systemarchitektur und Prozessmodell

Das entwickelte System lässt sich präzise als KI-gestützter Agent klassifizieren und direkt auf das im Grundlagenkapitel (siehe Abschnitt 2.4) vorgestellte Fünf-Komponenten-Modell nach Lanham abbilden. Die Gesamtarchitektur folgt einem serviceorientierten Ansatz, bei dem eine zentrale Orchestrierungsschicht (`main.py`) den sequentiellen Datenfluss steuert. Die folgenden Unterkapitel stellen die fünf Hauptkomponenten des Agenten vor und erläutern ihre Rolle in diesem sequentiellen Prozessmodell.

6.2.1 User Interface Layer

Diese Komponente (`main.py` und Streamlit) ist für die Entgegennahme des initialen Eingabeparameters - der GitHub Repository-URL - verantwortlich.

Durch die Bestätigung des Nutzers wird der in der Orchestrierungsschicht definierte Prozess angestoßen, der die **Planung (Planning)** des Agenten abbildet. Bei erfolgreicher Generierung

präsentiert die UI die validierte Pipeline und stellt einen Button zur Initiierung des Deployments (der Pull-Request-Erstellung) bereit.

6.2.2 Repository Analyzer

Diese Komponente (analytischer Teil des `github_client.py`) agiert als „Sensor“ des Agenten. Sie nimmt die Repository-URL entgegen und ist dafür verantwortlich, die „Umgebung“ (das Repository) wahrzunehmen. Sie extrahiert gezielt die im **Data Understanding** (siehe Abschnitt 4) als relevant identifizierten Dateien (z.B. *mta.yaml*, *package.json*) sowie die Anwendungsstruktur. Dabei gleicht der Analyzer die über die GitHub-API eingelesenen Dateinamen mit einer statischen Liste ab (siehe Quelltext 1).

Diese Dateien bilden das **Wissen und Gedächtnis (Knowledge/Memory)**, das als kontextbezogenes Kurzzeitgedächtnis für die Dauer der Pipeline-Generierung dient und an die nächste Komponente weitergegeben wird.

6.2.3 AI Core Integration Layer (LLM-Client)

Diese Komponente (`sap_ai_client.py`) bildet die kognitive Zentrale des Agenten. Sie nimmt die extrahierten Dateiinhalte (das Wissen) entgegen und konstruiert daraus einen detaillierten System Prompt. Dieser Prompt instruiert das über SAP AI Core angebundene LLM und definiert das **Profil (Persona)** des Agenten - seine Rolle, sein Ziel und die spezifischen Anforderungen an die zu generierende Pipeline.

Das Ergebnis ist die vom LLM generierte CI/CD-Pipeline in Form einer *workflow.yaml*-Datei, die an die nächste Komponente weitergeleitet wird.

6.2.4 Pipeline Validator

Der Validator (`validator.py`) repräsentiert die Fähigkeit des Agenten zum **Schlussfolgern und Bewerten (Reasoning/Evaluation)**. Unmittelbar nach der Generierung durch das LLM prüft diese Komponente die Pipeline als interne Qualitätssicherung. Sie führt eine deterministische YAML-Syntaxprüfung. Diese Prüfung agiert blockierend, dass heißt, sollte die Syntaxprüfung fehlschlagen, wird der Nutzer informiert und der Prozess neu gestartet. Es folgt eine regelbasierte Best-Practice-Analyse (z.B. Testing, CDS Model Validierung) durch und berechnet einen Qualitäts-Score. Dieser Schritt stellt sicher, dass nur syntaktisch korrekte und qualitativ ausreichende Artefakte an den Nutzer präsentiert oder weiterverarbeitet werden. Sollte die Pipeline die Kriterien nicht erfüllen, wird der Nutzer informiert und der Deployment-Prozess neugestartet.

6.2.5 Deployment Manager

Der Deployment Manager (ausführende Teil des `github_client.py`) stellt den „Akteur“ des Agenten dar und ist für die **Aktionen und Werkzeuge (Actions/Tool Use)** verantwortlich. Nach erfolgreicher Validierung durch den Pipeline Validator und auf explizite Anforderung des Nutzers über den User Interface Layer nutzt diese Komponente die GitHub-API, um in der Umgebung (dem Repository) zu handeln: Sie erstellt einen neuen Feature-Branch, committet die generierte *workflow.yml*-Datei in das *.github/workflows*-Verzeichnis und eröffnet einen Pull Request. Dabei arbeitet der Deployment Manager mit dem GitHub-Token, welcher über `contents` und `pull_request` Rechte verfügt; Branch-Naming erfolgt timestamp-basiert. Wichtig für die Nutzung des GitHub-Tokens, dass das Profil auf Basis der Token gebaut wurde, ebenfalls über die Zugriffsrechte auf das Repository verfügt.

Die Pipeline ist dabei, wie im System Prompt des Agenten verankert, so konfiguriert, dass sie bereits auf die Erstellung der Pull Requests reagiert. Dies stößt unmittelbar den Build- und Test-Job an (Smart-Trigging), wodurch die generierte Pipeline sich selbst sowie die CAP-Anwendung im Kontext des Ziel-Repositories validiert. Das finale Deployment auf die Cloud Foundry Runtime erfordert somit nur noch die Freigabe und das Zusammenführen (Mergen) der Pull Request.

6.3 Modellierungsentscheidungen und Alternativen

Aufbauend auf der Systemarchitektur werden in diesem Abschnitt die zentralen Design-Entscheidungen für die gewählten Technologien und Plattformen dargelegt und begründet.

6.3.1 Auswahl des Sprachmodells

Es wurde sich bewusst, wie bereits in Abschnitt 6.1 angeschnitten, gegen das Trainieren und Entwickeln eines eigenen Modells entschieden. Für ein eigenständiges Modelltraining wären umfangreiche und domänenspezifisch gelabelte Daten erforderlich, die im betrachteten Kontext nicht vorliegen. Darüber hinaus wäre der hierfür notwendige technische Aufwand - insbesondere hinsichtlich Rechenressourcen, Infrastruktur und Hyperparameteroptimierung - weder wirtschaftlich noch inhaltlich gerechtfertigt. Dies gilt umso mehr, als die Arbeit primär auf einen Proof-of-Concept abzielt und damit nicht den Anspruch verfolgt, ein vollständig domänenoptimiertes Modell zu entwickeln.

Stattdessen fiel die Wahl auf ein vortrainiertes Large Language Model (LLM). Im Rahmen der Evaluierung verschiedener LLMs, darunter GPT-4 und GPT-5, Claude Sonnet 2.5 sowie Gemini 2.5, zeigte sich, dass die generativen Ergebnisse hinsichtlich Strukturqualität, Konsistenz und syntaktischer Korrektheit weitgehend vergleichbar waren. Da somit kein Modell einen klaren Leistungs- oder Qualitätsvorteil aufwies, fiel die Wahl auf GPT-5 als Basismodell, da dieses auf einer aktuelleren Datengrundlage basiert und eine stabilere Kontextverarbeitung erwarten lässt.

6.3.2 Plattformwahl: SAP AI Core

Für das in dieser Arbeit entwickelte System wurde SAP AI Core als Plattform zur Integration des LLM ausgewählt. Da der geplante Einsatz im SAP-Umfeld stattfindet, ist davon auszugehen, dass interne bzw. potenziell vertrauliche Daten verarbeitet werden. Die Nutzung von SAP AI Core stellt sicher, dass alle Datenflüsse innerhalb einer kontrollierten Unternehmensumgebung verbleiben und damit etablierte Datenschutz- und Compliance-Vorgaben eingehalten werden können.

Ein zusätzlicher praktischer Vorteil ergibt sich aus der vorhandenen SAP-Lizenzierung: Die Nutzung der über AI Core angebotenen LLM-Deployments verursacht keine direkten Zusatzkosten. Dies reduziert den administrativen Aufwand und ermöglicht einen direkten Fokus auf die Umsetzung der eigentlichen Lösung.

Grundsätzlich wären alternative Ansätze denkbar gewesen (z. B. eigenes Hosting oder die Nutzung externer Cloud-LLMs). Diese hätten allerdings zusätzliche Aufwände verursacht, etwa hinsichtlich Infrastrukturbetrieb, Sicherheitsfreigaben oder abweichender Kostenmodelle, ohne für den hier betrachteten Anwendungsrahmen erkennbare Vorteile zu bieten.

Entsprechend erweist sich SAP AI Core als pragmatische Wahl, da es den organisatorischen Rahmenbedingungen entspricht und gleichzeitig einen sicheren und technisch eindeutigen Integrationspfad für das gewählte Sprachmodell bietet.

6.3.3 CI/CD-Plattform: GitHub Actions

Die für die Pipelines ausgewählte Plattform ist GitHub Actions. Diese Entscheidung basiert auf dessen nahtlosen Integration in das User Interface von GitHub. Mittels dieser können Entwickler:innen die generierten Workflows im Repository direkt einsehen und die Jobverwaltung zentral steuern. Alternative CI/CD-Plattformen wie Jenkins oder GitLab CI wurden evaluiert, jedoch zeigte sich, dass diese entweder eine komplexere Konfiguration erfordern oder nicht die gleiche Tiefe an nativer Integration in den GitHub-Workflow bieten. Ebenfalls

erwies sich GitHub Actions als geeignet, da die Anbindung und Konfiguration direkt mittels der GitHub-API ermöglicht wird.

6.3.4 Programmiersprache und Technologie-Stack

Als Implementierungssprache wurde Python gewählt, da die Anbindung an SAP AI Core und die GitHub-API durch ein robustes Bibliotheks-Ökosystem vereinfacht wurde.

Für das User Interface wurde auf die Python-Bibliothek Streamlit gesetzt. Die Wahl erfolgte aufgrund der engen Python-Integration und der Möglichkeit, interaktive Prototypen mit geringem Implementierungsaufwand zu entwickeln. Im Vergleich zu Alternativen wie Flask, Django oder React erfordert Streamlit keine separate Frontend-Logik, wodurch der Fokus auf die Kernfunktionalität des Agenten gelegt werden konnte. [25]

Auch auf Implementierungsebene wurde der deterministische Ansatz fortgeführt. Es wurde sich bewusst gegen eine komplexe Mustererkennung für den Repository Analyzer (z.B. mittels Regex [26]) entschieden. Da die Benennung der relevanten Dateien aufgrund der CAP-Namenskonventionen klar strukturiert und vorhersehbar sind, bietet der einfache Listenabgleich mit einer statischen „Allow-List“ eine höhere Effizienz und geringere Fehleranfälligkeit.

6.3.5 Prompt Engineering und Kontext-Steuerung

Eine zentrale Modellierungsentscheidung liegt im detaillierten System-Prompt (siehe Anhang A), der dem LLM im AI Core Integration Layer übergeben wird. Dieser ist nicht als einfache Anfrage (Query) zu verstehen, sondern als primäres Instrument zur Steuerung, Einschränkung und Wissensinjektion des Agenten.

Das Prompt Engineering verfolgt dabei mehrere strategische Ziele: Es erzwingt durch strikte Anweisungen (REQUIREMENTS, OUTPUT) ein deterministisches Ausgabeformat (valides YAML), das für die maschinelle Verarbeitung durch den Pipeline Validator essenziell ist. Gleichzeitig erzwingt es über PRODUCTION-GRADE FEATURES die Einhaltung von Best Practices, wie Multi-Stage-Jobs oder „Smart Triggering“, also der direkten Ausführung von Build und Test bei Erstellung der Pull-Request. Dieser statische Rahmen wird schließlich durch den dynamischen Teil (`{{files_section}}`) kontextualisiert, indem das im Repository Analyzer extrahierte Wissen (Anwendungsstruktur, *mta.yaml* und *package.json*) zur Anpassung an die spezifische CAP-Anwendung genutzt wird.

Der System-Prompt transformiert das generische LLM somit von einem Textgenerator zu einem spezialisierten Werkzeug. Er agiert als statische Wissensbasis und regelbasierte Einschränkung, die das LLM in einen deterministischen und qualitativ hochwertigen Generator für CI/CD-Pipelines überführt.

6.4 Grenzen und Herausforderungen

Die Grenzen des Systemes lassen sich definitiv auf die fehlende Lernfähigkeit zurückführen. Da der Agent nicht in der Lage ist, aus vergangenen Generierungsprozessen zu lernen sowie Feedback vom Nutzer zu empfangen und bearbeiten, bleibt er auf die initiale Prompt- und Regelbasis beschränkt und schließt damit die Vorteile einer Feedback-Loop komplett aus. Dies limitiert die Anpassungsfähigkeit und Optimierungspotenziale des Systems im Vergleich zu lernenden Modellen erheblich.

Direkt an diese fehlende Lernfähigkeit knüpft sich der hohe manuelle Wartungsaufwand. Da der Agent nicht in der Lage ist, sich selbstständig an neue Anforderungen oder Änderungen in den CAP-Anwendungen oder Nutzeranforderungen anzupassen müssen diese manuell in den System-Prompt oder die Validierungsregeln eingepflegt werden.

6.5 Zusammenfassung der Modellierung und Übergang zur Evaluation

Zusammenfassend wurde in dieser Phase ein **regelbasierter, kontextadaptiver KI-Agent** konzipiert und modelliert. Die Systemarchitektur (Abschnitt 6.2) setzt die im Grundlagenkapitel definierten theoretischen Komponenten eines Agenten in einen serviceorientierten Prozess um. Die Modellierungsentscheidungen (Abschnitt 6.3) favorisierten dabei bewusst einen deterministischen und kontrollierten Ansatz (statische Regeln, detailliertes Prompt Engineering) gegenüber einem adaptiven, lernenden Modell.

Das Ergebnis ist ein voll funktionsfähiger Prototyp, der die im Data Understanding (Abschnitt 4) identifizierten Merkmale von CAP-Anwendungen interpretieren und in valide CI/CD-Workflows überführen kann.

Nachdem das Modell nun konzeptionell definiert und implementiert ist, folgt im nächsten Kapitel, die Evaluation. Diese prüft, inwieweit das Modell die im Business Understanding (Abschnitt 3) formulierten Ziele - die Generierung korrekter und praxistauglicher Pipelines - im Rahmen der 5 CAP-Anwendungen (Abschnitt 4) tatsächlich erfüllt.

7 Evaluation

Mit Abschluss des Modelings rückt im CRISP-DM-Prozess die Evaluationsphase in den Vordergrund. Während die vorherigen Schritte überwiegend technischer Natur waren, erfordert die Evaluation erneut eine enge Verzahnung zwischen technischer Perspektive und fachlichem Kontext. Ziel ist es, den entwickelten KI-Agenten nicht nur hinsichtlich seiner technischen Korrektheit zu überprüfen, sondern vor allem zu bewerten, ob er den im Business Understanding formulierten Anforderungen tatsächlich gerecht wird. Dabei wird der gesamte Entstehungsprozess - einschließlich Datenbasis, Datenqualität, Modellierungsentscheidungen und der generierten Ergebnisse - erneut reflektiert.

Im Mittelpunkt steht die Frage, inwiefern das System in der Lage ist, valide, funktionsfähige und praxistaugliche CI/CD-Pipelines für SAP-CAP-Anwendungen zu erzeugen. Da diese Pipelines produktionsnahe Artefakte darstellen, spielt neben inhaltlicher Richtigkeit insbesondere die tatsächliche Ausführbarkeit eine zentrale Rolle. Auf Grundlage dieser Bewertung wird schließlich entschieden, ob der Prototyp grundsätzlich für ein Deployment - also den Betrieb im regulären Entwicklungsumfeld - geeignet wäre.

7.1 Evaluationsmethode: Human-as-a-Judge

Für die Bewertung der generierten Pipelines wird ein Human-as-a-Judge-Verfahren eingesetzt. Die Ergebnisse werden dabei nicht ausschließlich anhand quantitativer Kennzahlen beurteilt, sondern vorrangig durch die fachliche Einschätzung von Personen, die im konkreten Anwendungskontext technisch kompetent und organisatorisch involviert sind. Dieser Ansatz ist insbesondere dann sinnvoll, wenn keine gelabelten Vergleichsdaten vorliegen oder die Qualität eines Artefakts nicht hinreichend über numerische Metriken erfasst werden kann.[27]

Die Bewertung wurde durch mich vorgenommen und durch erfahrene Entwickler:innen aus der Abteilung sowie weitere Kolleg:innen innerhalb der SAP fachlich unterstützt.

Ziel war es sicherzustellen, dass die generierten Workflows nicht nur syntaktisch valide, sondern vor allem im jeweiligen Projektkontext tatsächlich einsetzbar sind.

Die Entscheidungsbasis beruht dabei auf den im Business Understanding definierten Kriterien:

- Funktionale Vollständigkeit: Alle wesentlichen Schritte - insbesondere Build-, Test- und Deployment-Prozesse - müssen enthalten sein.
- Syntaktische Korrektheit: Die Pipeline muss formal gültig sein, insbesondere im Hinblick auf die YAML-Struktur.
- Praktische Effizienz: Die Pipeline sollte ohne unnötige Build- oder Deployment-Schritte auskommen und einen schlanken, nachvollziehbaren Ablauf sicherstellen.

Ein rein quantitatives Vorgehen - beispielsweise über Ähnlichkeitsmaße wie S-BERT oder ROUGE - wäre in diesem Kontext nicht zielführend. Da für die betrachteten Repositories keine Referenz-Pipelines existierten, hätte ein Vergleich mit hypothetischen Zielformaten keine belastbare Aussagekraft. Zudem erfassen solche Metriken nicht, ob eine Pipeline tatsächlich fehlerfrei ausgeführt werden kann, was in der Praxis ein gesonderter Validierungsschritt ist. [27] Entscheidend ist daher die reale Durchführbarkeit, die sich zuverlässig nur durch die Kombination aus fachlicher Expertise, Ausführungserfahrung und projektspezifischem Kontextwissen beurteilen lässt.

Auch LLM-as-a-Judge wurde geprüft, aber bewusst ausgeschlossen. LLMs können zwar zur theoretischen Bewertung von Code herangezogen werden, sie können jedoch nicht beurteilen, ob eine Konfiguration in der Praxis funktioniert. Um die Qualität einer CAP-Build- und Deployment-Pipeline einzuschätzen, ist Fachwissen über die SAP-spezifischen Werkzeuge und Prozesse nötig. Ein LLM kann nicht prüfen, ob die Pipeline in der realen Zielumgebung - in unserem Falle die Cloud Foundry Runtime - tatsächlich lauffähig wäre. Da LLMs keine Ausführungsergebnisse interpretieren können und zudem ein inhärentes Halluzinationsrisiko besteht, wären Fehlbewertungen wahrscheinlich [28]. Darüber hinaus entsteht eine methodische Zirkularität, wenn ein Modell Ergebnisse bewertet, die von demselben (oder einem vergleichbaren) Modell erzeugt wurden - eine unabhängige Validierungsinstanz fehlt. Entsprechend war der Rückgriff auf menschliche Expertise nicht nur pragmatisch, sondern methodisch geboten.[29], [30]

7.2 Durchführung

Wie im Data Understanding beschrieben, wurden für die Evaluation fünf exemplarische CAP-Anwendungen aus unterschiedlichen Konstellationen (Variationen in Node-Versionen, Datenbanken, UI-Stack, etc.) herangezogen. Ziel war nicht, eine statistische Generalisierbarkeit zu erreichen, sondern die Leistungsfähigkeit des KI-Agenten unter realistischen und variierenden Rahmenbedingungen zu untersuchen.

Die Pipelines wurden in jeweils rund 40-50 Sekunden generiert und anschließend ausgeführt. Dabei zeigte sich, dass alle generierten Workflows erfolgreich lauffähig waren; die notwendigen Anpassungen betrafen dabei fast ausschließlich die Repository-Struktur und deren Datenqualität, wie bereits in Data Understanding und Data Preparation beschrieben, und nicht die generierten Pipeline-Artefakte selbst.

Die frühere intensive Anpassung des Prompts war vor allem in der frühen Entwicklungsphase nötig. Nach dessen Stabilisierung traten nur noch punktuelle Modifikationen auf, etwa bei Projekten mit nicht-standardisierten Strukturen. Insgesamt zeigten die generierten Pipelines ein stabiles und konsistentes Format und repräsentierten gute Entwicklungspraktiken im CAP-Umfeld.

7.3 Bewertung

Die Ergebnisse können für alle fünf Testfälle als positiv gewertet werden. Der Agent war dazu in der Lage, reproduzierbar funktionsfähige CI/CD-Pipelines zu erzeugen, die im Anschluss erfolgreich die CAP-Anwendungen auf Cloud Foundry deployen konnten. Die generierten Workflows waren syntaktisch korrekt (Validiert durch den Pipeline Validator), praktikabel strukturiert und integrierten best-practice Komponenten wie Build- und Test-Steps sowie MTAR-Erzeugung.

Dieser Erfolg ist das Ergebnis eines iterativen Prozesses. Da LLMs nicht-deterministisch arbeiten, wurden die Generierungen pro Repository mehrfach ausgeführt. In frühen Testläufen traten dabei kleinere Mängel auf, beispielsweise das Einspielen der falschen Node.js-Version. Durch iteratives Prompt Engineering (Fine-Tuning am System-Prompt) konnten diese Abweichungen jedoch eliminiert werden. Die finalen Pipelines in der Evaluation wurden daraufhin

bei der ersten Generierung korrekt und mit durchgehend positiver Performance in allen drei Bewertungskriterien erzeugt.

Die Einbindung menschlicher Expertise erwies sich dabei als sinnvoll und notwendig: Nur durch die Kombination aus automatisierter Generierung und fachlicher Bewertung konnte sichergestellt werden, dass die Artefakte nicht nur formal korrekt erscheinen, sondern auch den realen Anforderungen des Zielsystems gerecht werden. Fachfeedback half vor allem bei der Einschätzung, ob die generierten Schritte sinnvoll priorisiert wurden und ob potenzielle Optimierungen - etwa im Hinblick auf Build-Performance oder Deployment-Zeitpunkte - denkbar wären.

Einschränkend muss festgehalten werden, dass die Evaluation auf einer bewusst klein gehaltenen, exemplarischen Datenbasis von fünf Anwendungen beruht. Die positiven Ergebnisse sind daher nicht als statistisch repräsentative Studie zu werten, sondern als ein Nachweis der grundsätzlichen Machbarkeit und der Validierung des Lösungsansatzes (Proof-of-Concept) im Rahmen der zur Verfügung stehenden Projektdaten.

7.4 Entscheidung über Deployability

Im Sinne des CRISP-DM-Prozesses erlaubt die Evaluation eine fundierte Aussage darüber, ob sich der entwickelte Prototyp grundsätzlich für den Einsatz im realen Entwicklungsbetrieb eignet. Aufgrund der stabilen Ergebnisse, der erfolgreichen Ausführbarkeit der Pipelines und des positiven Feedbacks der beteiligten Entwickler:innen kann die grundlegende Deployability bejaht werden. Für den produktiven Einsatz wären allerdings weitere Validierungszyklen empfehlenswert, insbesondere mit einer größeren und diversifizierteren Datenbasis (d.h. mehr CAP-Anwendungen), sowie eine Erweiterung des Prompts um zusätzliche Randfallregelungen.

8 Deployment

Die letzte, finale Phase im CRISP-DM-Prozess ist die Operationalisierung des Modells, auch Deployment genannt. Hierzu wird das vorher erstellte Modell in die IT-Infrastruktur integriert, so dass es durchgehend in Betrieb ist. Was dabei nicht vergessen werden darf, ist die kontinuierliche Überwachung einerseits der Verfügbarkeit, andererseits der Performance des Modells. Ziel dieser Phase war es, den zuvor entwickelte Prototyp im Unternehmen bereitzustellen.

8.1 Zielplattform und Bereitstellung

Als Zielplattform wurde die SAP Cloud Foundry (CF) gewählt. Als Runtime innerhalb der BTP ermöglicht sie eine einfache Bereitstellung direkt aus dem Quellcode, ohne dass Server verwaltet werden müssen. Die bereits im Modeling beschriebene Systemarchitektur, bei der die Streamlit-App als Frontend dient, wurde auf dieser Plattform abgebildet.

Die technische Bereitstellung des Python-Projekts erfolgte über den Standard-`'cf push'`-Befehl. Gesteuert wurde dieser Prozess durch zwei wesentliche Dateien:

- *manifest.yml*: Diese Konfigurationsdatei definiert die Rahmenbedingungen für die Anwendung. Sie legt den Namen (`sap-cap-pipeline-generator`), den Speicher (512M) und den `python_buildpack` fest. Der entscheidende Eintrag ist der `command`, der Streamlit anweist, auf dem dynamisch von CF zugewiesenen `$PORT` zu lauschen.
- *requirements.txt*: Enthält alle Python-Abhängigkeiten (wie `streamlit` und `requests`), die der Buildpack automatisch bei der Bereitstellung installiert.

8.2 Konfigurations- und Sicherheitsmanagement

Ein zentraler Aspekt der Operationalisierung ist die strikte Trennung von Code und Konfiguration. Sensible Daten, insbesondere die Credentials für den SAP AI Core und XSUAA-Endpunkte, dürfen nicht im Quellcode gespeichert werden.

Lokale Konfigurationsdateien (wie die `.env`) wurden daher über die `.cfignore`-Datei explizit von der Bereitstellung ausgeschlossen.

Alle für den Betrieb notwendigen Credentials und Endpunkte wurden stattdessen nach dem Deployment als Cloud Foundry Environment Variables (Umgebungsvariablen) gesetzt.

Dies geschah über den `cf set-env`-Befehl. Dieser Ansatz stellt sicher, dass der Code sicher im Git-Repository gespeichert werden kann und Konfigurationen flexibel im Betrieb geändert werden können.

8.3 Überwachung im Betrieb

Um die von CRISP-DM geforderte kontinuierliche Überwachung sicherzustellen, wurden die folgenden nativen Werkzeuge der Cloud Foundry genutzt:

- Die Verfügbarkeit (Status der App) wurde mittels `'cf apps'` überwacht.
- Die Performance und Fehleranalyse (z.B. bei der API-Kommunikation mit AI Core) erfolgte durch die Analyse der Echtzeit-Logs mittels `'cf logs sap-cap-pipeline-generator --recent'`.

Nach einem finalen Neustart durch den Befehl `cf restart`, um die Umgebungsvariablen zu laden, war der Prototyp erfolgreich operationalisiert und für die Zielgruppe erreichbar.

9 Fazit und Ausblick

Ziel dieser Arbeit war es zu untersuchen, inwieweit sich ein KI-Agent zur automatisierten Generierung von CI/CD-Pipelines für CAP-Anwendungen realisieren lässt. Ausgangspunkt war die Beobachtung, dass das manuelle Erstellen solcher Pipeline-Konfigurationen zeit- aufwändig und fehleranfällig ist und damit ein relevantes Automatisierungspotenzial besteht. Auf Basis des CRISP-DM-Prozesses wurde hierfür ein vollständiger Prototyp entwickelt, der CAP-Repositories analysiert und automatisch eine lauffähige Pipeline in Form einer *github/workflows/deploy.yml* bereitstellt.

Im Ergebnis zeigt sich, dass der Ansatz grundsätzlich funktioniert: Der Agent war in allen Testfällen in der Lage, ein valides Pipeline-Artefakt zu generieren. Die Qualität der Ergebnisse wurde im Rahmen einer Human-as-a-Judge-Evaluation beurteilt und anhand der Kriterien funktionale Vollständigkeit, syntaktische Korrektheit sowie praktische Effizienz bewertet. Besonders positiv fällt auf, dass nur vergleichsweise geringe Nacharbeiten nötig waren. Anpassungen erfolgten hauptsächlich in frühen Entwicklungsphasen (vor allem am System-Prompt), weniger an den generierten Pipelines selbst. Damit konnte gezeigt werden, dass ein generativer Ansatz die Erstellung von CI/CD-Konfigurationen spürbar vereinfachen und konsistenter gestalten kann.

Gleichzeitig wurde deutlich, dass der Erfolg maßgeblich von einer guten Datenbasis abhängt. Die getesteten CAP-Projekte wiesen teilweise unvollständige oder uneinheitliche Strukturen auf, die zunächst bereinigt oder ergänzt werden mussten. Insofern ist nicht allein der Agent ausschlaggebend, sondern ebenso der Reifegrad des jeweiligen Projekts. Darüber hinaus bildet die qualitative Evaluation valide erste Einschätzungen ab, ersetzt jedoch keine systematische quantitative Bewertung - insbesondere im Hinblick auf die Generalisierungsfähigkeit bei einer größeren Anzahl und Varianz von Projekttypen.

Ausblickend eröffnet der Ansatz interessante Möglichkeiten. Denkbar wäre etwa, den Agenten enger in die Entwicklungsumgebung zu integrieren, sodass Pipeline-Anpassungen automatisiert vorgeschlagen oder direkt validiert werden können. Auch eine Kombination mit anderen Agenten - etwa zur Codeanalyse, Security-Checks oder Projekt-Bootstrap-Generie-

rung - erscheint vielversprechend. Parallel ließe sich der Fokus erweitern: Statt ausschließlich CAP-Projekte zu betrachten, könnten auch allgemeine Node.js- oder Multi-Service-Setups unterstützt werden.

Darüber hinaus existieren inzwischen alternative Werkzeuge, die in eine ähnliche Richtung gehen: GitHub-basiert etwa Copilot oder Cline, die kontextsensitiv Entwicklungsaufgaben unterstützen und Code-Artefakte erzeugen können; ebenso Cursor oder klassische LLMs wie ChatGPT oder Gemini. Im Gegensatz zu diesen generalisierten, oft rein textbasierten Assistenten wurde in dieser Arbeit ein hochspezialisierter Agent entwickelt. Der entscheidende Mehrwert dieses Ansatzes liegt in der Kombination aus Domänenspezialisierung und Steuerbarkeit: Der Agent ist durch den detaillierten System-Prompt und die Validierungsregeln exakt auf den Nischen-Anwendungsfall (SAP CAP auf Cloud Foundry) und dessen Best Practices (z.B. MTAR) abgestimmt. Er agiert als deterministisch gesteuerter Prozess (Analyse, Injektion, Validierung) und nicht als reiner Textgenerator. Diese Aspekte, kombiniert mit der sicheren Einbettung in die Unternehmenslandschaft (SAP AI Core), bieten Vorteile in puncto Sicherheit, Reproduzierbarkeit und fachlicher Korrektheit, die generische Tools nicht leisten können.

Insgesamt zeigt die Arbeit, dass ein KI-gestützter Agent zur Generierung von CI/CD-Pipelines für CAP-Anwendungen technisch umsetzbar ist und echten Mehrwert bietet. Zwar bleibt weiterer Entwicklungsaufwand notwendig - insbesondere in puncto Skalierbarkeit, einer quantitativen Evaluation auf einer breiteren Datenbasis sowie einer automatisierten Test-Validierung der generierten Artefakte - dennoch verdeutlichen die Ergebnisse das Potenzial, repetitive Entwicklungsaufgaben gezielt zu automatisieren und so Entwickler:innen zu entlasten. Damit kann die Lösung ein Baustein sein, um die Software-Entwicklung effizienter und konsistenter zu gestalten, ohne den Menschen aus dem Entscheidungsprozess zu verdrängen.

A System-Prompt

Generate a production-ready GitHub Actions workflow YAML for SAP CAP deployment to {target_env}.

REQUIREMENTS:

- Return ONLY the YAML content, no explanations
- No markdown code blocks
- Start directly with 'name:'

PRODUCTION-GRADE FEATURES:

1. Separate CI/CD jobs:

- Job 1: build-test (runs on all PRs and pushes)
- Job 2: deploy (only on main ,branch, depends on build-test)

2. Comprehensive testing:

- npm ci (clean install)
- npx cds compile srv --to csn (validate CDS models)
- npm test (if test script exists)
- npm run build (CDS build)

3. Robust MTAR handling:

- Generate MTAR with consistent naming in build-test job
- Upload MTAR as artifact
- In deploy job:
 - Download MTAR artifact - not @sap/mbt -> npx -y -p mbt mbt

build...

- Dynamically find MTAR file (do not rely on fixed path or commit SHA)

- Deploy job must be restartable independently of build-test reruns

4. Smart triggering:

- on: push: branches: [main] AND pull_request: branches: [main]
- Deploy job runs only on push to main

Quelltext 2 – Vollständiger System-Prompt für dieAI Core Integration Layer, Teil 1

5. Error handling:

- Use 'set -e' in multi-line scripts
- Chain commands with && for early failure detection
- Proper working directory management

6. Professional structure:

- Clear job names and step descriptions
- Proper Node.js version matrix or fixed version
- ****Install Cloud Foundry CLI directly in deploy job****
 - Do not rely on deprecated cloudfoundry/cf-cli-action
 - Download official v8 Linux binary
 - Extract archive and move the `cf` binary from its subdirectory into /usr/local/bin/ so it is in PATH
 - Verify installation with `cf version`
 - Install Multiapps plugin in deploy job after CLI setup
- Secret management for CF credentials (CF_API, CF_ORG, CF_SPACE, CF_USERNAME, CF_PASSWORD)

7. PR-Integration:

- build-test job runs on PRs to validate changes before merging

8. Deploy job (always do as referenced here!):

- Very Important!: use the following guide as reference for ubuntu/Linux setup from the official docs:

Linux installation

To install the cf CLI on Debian and Ubuntu-based Linux distributions:

Add the Cloud Foundry Foundation public key and package repository to your system by running:

```
wget -q -O - [https://packages.cloudfoundry.org/debian/cli.
cloudfoundry.org.key](https://packages.cloudfoundry.org/debian/cli.
cloudfoundry.org.key) \
    | sudo gpg --dearmor -o /usr/share/keyrings/cli.cloudfoundry.org.gpg

echo "deb [signed-by=/usr/share/keyrings/cli.cloudfoundry.org.gpg] \
[https://packages.cloudfoundry.org/debian](https://packages.
cloudfoundry.org/debian) stable main" \
    | sudo tee /etc/apt/sources.list.d/cloudfoundry-cli.list
```

Update your local package index by running:

```
sudo apt-get update
```

To install cf CLI v7, run:

```
sudo apt-get install cf7-cli
```

To install cf CLI v8, run:

```
sudo apt-get install cf8-cli
```

IMPORTANT:

- `npm ci` will fail if package.json and lockfile are out of sync.
Update lockfile before continuing.

OUTPUT:

- YAML only, starting with 'name:'
- Keep the **build-test** commands exactly as before
- Adjust **deploy job steps** to:
 - Dynamically detect MTAR from artifact
 - Install CF CLI correctly from official binary with proper path handling
 - Install Multiapps plugin
 - Use CF secrets correctly
 - Ensure deploy job is restartable

Use the following files for guidance and orientation:{files_section}

Quelltext 4 – Vollständiger System-Prompt für dieAI Core Integration Layer, Teil 3

Literaturverzeichnis

- [1] D. C. E. Winn, *Cloud foundry*, First edition. Beijing: O'Reilly, 2017.
- [2] P. Chapman u. a., „CRISP-DM 1.0: Step-by-step data mining guide“, 2000. Zugegriffen: 27. August 2025. [Online]. Verfügbar unter: <https://www.kde.cs.uni-kassel.de/wp-content/uploads/lehre/ws2012-13/kdd/files/CRISPWP-0800.pdf>
- [3] I. Weber und L. Zhu, Hrsg., *DevOps*, 1. Auflage. Boston: Addison-Wesley, 2015.
- [4] J. Humble, *Continuous delivery*, 7th printing 2013. in A @Martin Fowler Signature Book. Upper Saddle River, NJ: Addison-Wesley, 2013.
- [5] N. Forsgren, *Accelerate*. Portland, Oregon: IT Revolution, 2018.
- [6] F. Zampetti, S. Geremia, G. Bavota, und M. Di Penta, „CI/CD Pipelines Evolution and Restructuring: A Qualitative and Quantitative Study“, in *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, IEEE, Sep. 2021, S. 471–482. doi: [10.1109/icsme52107.2021.00048](https://doi.org/10.1109/icsme52107.2021.00048).
- [7] G. Bajpai, *CI/CD Design Patterns*, 1st ed. Packt Publishing, 2024.
- [8] H. van Merode, *Continuous Integration (CI) and Continuous Delivery (CD)*, 1st ed. Berkeley, CA: Apress L. P., 2023.
- [9] T. Clark, *CI/CD unleashed*. in Apress pocket guides. Berkeley, CA: Apress, 2025.
- [10] Atlassian, „Continuous integration vs. delivery vs. deployment“. Zugegriffen: 3. November 2025. [Online]. Verfügbar unter: <https://www.atlassian.com/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment>
- [11] K. Kopecz, *Anwendungsentwicklung auf der SAP Cloud Platform*, 1. Auflage. in SAP PRESS. Bonn: Rheinwerk Verlag, 2021.

- [12] SAP SE, „Developing with the SAP Cloud Application Programming Model“. Zugegriffen: 9. November 2025. [Online]. Verfügbar unter: <https://help.sap.com/docs/btp/sap-business-technology-platform/developing-with-sap-cloud-application-programming-model-cap>
- [13] SAP SE, „SAP Cloud Application Programming Model (CAP) Documentation“. Zugegriffen: 9. November 2025. [Online]. Verfügbar unter: <https://cap.cloud.sap/>
- [14] H. Seubert, *SAP Business Technology Platform*, 2., aktualisierte und erweiterte Auflage. in Rheinwerk Publishing. Bonn: Rheinwerk Verlag, 2022.
- [15] M. Lanham, *AI agents in action*, [First edition]. Shelter Island, NY: Manning Publications, 2025.
- [16] „Was ist ein großes Sprachmodell (LLM)?“, 2024. Zugegriffen: 3. November 2025. [Online]. Verfügbar unter: <https://www.cloudflare.com/de-de/learning/ai/what-is-large-language-model/>
- [17] „What Are Large Language Models (LLMs)?“, Nov. 2023. [Online]. Verfügbar unter: <https://www.ibm.com/topics/large-language-models>
- [18] L. Bhowmik, *Machine learning with SAP*, 1st edition. in SAP Press. Boston (MA): Rheinwerk Publishing, 2021.
- [19] J. Alammari, *Hands-on large language models*, First edition. Beijing: O'Reilly, 2024.
- [20] I. Goodfellow, *Deep Learning*, 1st edition. in mitp Professional. Frechen: mitp, 2018.
- [21] RedHat, „Understanding Automation“. Zugegriffen: 13. Oktober 2025. [Online]. Verfügbar unter: <https://www.redhat.com/en/topics/automation>
- [22] R. Kergaßner, „IT-Automatisierung schafft Freiräume“, *Wirtschaftsinformatik & Management*, Bd. 4, Nr. 6, S. 20–23, Dez. 2012, doi: [10.1365/s35764-012-0190-2](https://doi.org/10.1365/s35764-012-0190-2).
- [23] O. Zeigermann, *Machine Learning*, 3. Auflage. Heidelberg: O'Reilly, 2024.
- [24] C. S. Shruthi H. Shetty Sumiksha Shetty und A. Rao, *Fundamentals and methods of machine and deep learning*. Beverly, MA: Wiley, 2022.

-
- [25] Streamlit, „Streamlit - A faster way to build and share data apps“. Zugegriffen: 30. Oktober 2025. [Online]. Verfügbar unter: <https://streamlit.io/>
- [26] Z. Nagy, *Regex Quick Syntax Reference*. Berkeley, CA: Apress L. P., 2018.
- [27] L. Zheng u. a., „Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena“. arXiv, 2023. doi: [10.48550/ARXIV.2306.05685](https://doi.org/10.48550/ARXIV.2306.05685).
- [28] L. Tunstall, *Natural language processing with transformers*, Revised edition. Beijing: O'Reilly, 2022.
- [29] M. Gao, X. Hu, J. Ruan, X. Pu, und X. Wan, „LLM-based NLG Evaluation: Current Status and Challenges“. arXiv, 2024. doi: [10.48550/ARXIV.2402.01383](https://doi.org/10.48550/ARXIV.2402.01383).
- [30] K. Schroeder und Z. Wood-Doughty, „Can You Trust LLM Judgments? Reliability of LLM-as-a-Judge“. arXiv, 2024. doi: [10.48550/ARXIV.2412.12509](https://doi.org/10.48550/ARXIV.2412.12509).